

Sherlock Internship Challenge

Build an AI System to Identify the Interview Candidate in Real Time

Background

Sherlock is an AI platform that detects fraud during live interviews conducted over Google Meet, Microsoft Teams, and Zoom.

Our fraud detectors (deepfake detection, voice cloning, behavioral analysis, etc.) must analyze the **candidate's** audio and video—not the interviewer or any other participant.

Today, candidate identification is one of the biggest challenges in our architecture.

For example:

- Candidate joins as **MacBook Pro**
- Candidate joins using a nickname
- Interviewer enters the wrong candidate name
- Multiple interviewers are present
- Candidate changes their display name
- Multiple observers join silently

We want Sherlock to automatically identify the candidate with the **highest possible confidence** in real time.

Your Challenge

Design and build a working prototype that automatically identifies the interview candidate during a live meeting.

You are free to use:

- AI models
- LLMs
- Computer vision
- Speech analysis

- Rule engines
- Agentic workflows
- Any open-source libraries
- Any cloud services

There is **no predefined solution**.

We want to see how you think.

Available Information

Assume your system has access to:

Participant Information

- Participant ID
- Display name
- Join/leave events
- Webcam on/off
- Screen share events

Audio

- Separate audio stream for every participant
- Speaking activity
- Speaking duration

Video

- Separate webcam stream for every participant

Transcript

- Speaker-attributed transcript

External Metadata

- Candidate name
- Candidate email
- Calendar invite
- Interview schedule

- Interviewer names
-

Requirements

Your prototype should:

- Automatically identify the candidate
 - Continuously update confidence during the interview
 - Handle incorrect names
 - Handle missing information
 - Handle ambiguous situations
 - Explain why it selected a participant
-

Deliverables

1. Working Demo

Submit a working prototype.

The implementation language and framework are your choice.

2. Short Demo Video (5–10 minutes)

Walk us through:

- Architecture
 - Approach
 - Demo
 - Trade-offs
 - What you'd improve next
-

3. GitHub Repository

Include:

- Source code
 - README
 - Setup instructions
 - Assumptions
-

4. Architecture Diagram

Explain your system.

5. Evaluation

Describe:

- How you tested your system
 - Edge cases
 - Accuracy
 - Limitations
-

Bonus Points

We love solutions that:

- ✓ Use multiple weak signals instead of relying on one rule.
 - ✓ Produce a confidence score.
 - ✓ Explain why a participant was selected.
 - ✓ Continue learning as more interview data becomes available.
 - ✓ Work in real time.
 - ✓ Gracefully handle uncertainty instead of making incorrect assumptions.
-

Evaluation Criteria

Category	Weight
Problem-solving ability	25%
Engineering quality	20%
AI/ML approach	20%
Product thinking	15%
Scalability	10%
Code quality	5%
Creativity	5%

Constraints

- You may use any AI models or APIs.
- You may use LLMs extensively.
- You may use open-source projects.
- You may build agents.
- You may use cloud infrastructure.
- There are no restrictions on implementation.

The only requirement is that the solution should be practical and demonstrate how Sherlock could identify the correct interview participant reliably in real time.

What We're Looking For

This challenge is intentionally open-ended.

We're not looking for the "correct" solution—we're looking for engineers who can tackle ambiguous, real-world problems using first-principles thinking.

The strongest submissions will likely combine multiple sources of evidence, reason under uncertainty, and produce a confidence score rather than relying on a single heuristic.

One important note: Don't optimize only for the final answer. Show us *how* your system reaches its conclusion. In production, explainability, confidence estimation, and graceful handling of ambiguity are just as important as raw accuracy.

Send your github link to priya@sherlock.sh .